

Tensor Indexing - Setter

- 1 [Introduction](#)
- 2 [Torch Setitem](#)
 - 2.1 [Slice](#)
 - 2.2 [Indexing Tensors](#)
 - 2.3 [Combine](#)
- 3 [Implementation Details](#)
 - 3.1 [General Validation](#)
 - 3.2 [Row-Major Interleaved Inputs, Output](#)
 - 3.2.1 [Validation & Output Spec](#)
 - 3.2.2 [Pipeline](#)
 - 3.2.3 [Reader Stage](#)
 - 3.2.4 [Writer Stage](#)
 - 3.2.5 [Pseudo code](#)
 - 3.2.6 [Memory Usage](#)
 - 3.3 [TILE Interleaved I/O](#)
 - 3.3.1 [Validation & Output Spec](#)
 - 3.3.2 [Reader Stage](#)
 - 3.3.3 [Writer Stage](#)
- 4 [Performance Evaluation](#)
 - 4.1 [Objective](#)
 - 4.2 [Methodology](#)
 - 4.3 [Testcases](#)
 - 4.4 [Runtime Benchmark](#)
 - 4.5 [Smoke testing](#)
- 5 [References](#)

Introduction

This document details our implementation of tensor indexing setter kernels on Tenstorrent's Wormhole architecture, leveraging the TT-Metal software stack. Our target operation reads a source tensor of update values and writes them into a destination tensor at positions determined by an index tensor. We focus on maximizing throughput under the Wormhole's memory hierarchy by carefully orchestrating DRAM transfers, interleaving, and on-chip buffering.

Torch Setitem

The `tttn.setitem` operation mimics PyTorch's tensor assignment semantics. This includes support for both **slicing** and **advanced indexing using indexing tensors**, following shape and broadcasting rules defined by PyTorch. Understanding how shapes are inferred and matched during these operations is critical when implementing or validating behavior involving `index_tensors`, `index_dims`, and `begins/ends/steps`.

This section breaks down each indexing mode individually, and then shows how they work in combination.

Slice

Slicing is the most common indexing mechanism. It allows selecting a contiguous region of a tensor along one or more dimensions using Python's slice syntax: `start:end:step`. In `tttn.setitem`, slices are specified using the `begins`, `ends`, and `steps` arrays.

Key behavior:

- A slice preserves the **rank** of the tensor.
- The size of a sliced dimension is `(end - start) // step`.

✓ Example

```
1 a = torch.zeros(4, 5)
2 a[1:3, :] = torch.ones(2, 5)
```

- `dim 0` is sliced from 1 to 3 → size 2
 - `dim 1` is fully sliced → size 5
 - **Set shape** = `(2, 5)`
-

Indexing Tensors

Indexing tensors (a.k.a. advanced indexing) allow you to select arbitrary (non-contiguous) positions in one or more dimensions by providing integer tensors or lists.

In `tttn.setitem`, indexing tensors are passed through the `index_tensors` list, and their corresponding dimensions are specified via `index_dims`.

Key behavior:

- All `index_tensors` must be **broadcastable** to a common shape.
- This broadcasted shape determines how many values are written.
- Each dimension in `index_dims` is replaced by the broadcasted shape.
- All non-indexed dimensions are sliced (or default to full range).
- **Final set shape** = `broadcast_shape + sliced_shape`.

✓ Example 1: 1D indexing

```
1 a = torch.zeros(4, 5)
2 a[torch.tensor([1, 3])] = torch.ones(2, 5)
```

- Indexing `dim 0` → shape (2,)
- Remaining dim 1 → shape 5
- **Set shape** = `(2, 5)`

✓ Example 2: Multi-dim indexing with broadcast

```
1 a = torch.zeros(3, 4, 6)
2
3 idx0 = torch.tensor([[0], [1]]) # shape (2,1)
4 idx1 = torch.tensor([1, 3])    # shape (2,)
5 a[idx0, idx1] = torch.ones(2, 2, 6)
```

- `index_dims` = `[0, 1]`
- Broadcasted index shape = `(2, 2)`
- Remaining dim 2 → shape 6
- **Set shape** = `(2, 2, 6)`

✓ Example 3: Scalar assignment with advanced indexing

```
1 a = torch.zeros(3, 3)
2 a[[0, 1], [1, 2]] = 42
```

- Index tensors broadcast to shape (2,)
- No remaining dims
- Set shape = `(2,)` → scalar `42` broadcast to 2 elements

✗ Invalid: Non-broadcastable indices

```
1 a = torch.zeros(4, 4, 4)
2 a[[0], [0, 1, 2], [1]] = b # ERROR!
```

- Index shapes = `(1,)`, `(3,)`, `(1,)` → cannot broadcast

-  Fix:

```
1 a[[0, 0, 0], [0, 1, 2], [1, 1, 1]] = b
```

Combine

`tttn.setitem` allows you to combine **slice-based indexing** and **indexing tensors**.

This pattern allows selecting structured blocks (via slices) and scattered indices (via indexing tensors) simultaneously.

Key behavior:

- All `index_tensors` are broadcast to a common shape `I`
- Dimensions in `index_dims` are replaced by `I`
- Sliced (non-indexed) dimensions form shape `S`
- **Final set shape** = `I + S`
- RHS `value` must match or be broadcastable to this shape

Example

```
1 a = torch.zeros(4, 3, 5)
2
3 idx0 = torch.tensor([0, 2])      # shape (2,)
4 idx2 = torch.tensor([[1], [3]]) # shape (2,1)
5 index_dims = [0, 2]
6
7 # Slicing dim 1: begin=0, end=3, step=1
8 begins = [0, 0, 0]
9 ends = [4, 3, 5]
10 steps = [1, 1, 1]
11
12 a[idx0, :, idx2] = torch.ones(2, 2, 3)
```

- Index tensors broadcast to shape `(2, 2)`
- Sliced dim 1: size 3
- Final set shape: `(2, 2, 3)`

Example: Partial indexing

```
1 a = torch.zeros(4, 5, 6)
2 idx0 = torch.tensor([0, 2])
3 a[idx0, 1:4, :] = torch.ones(2, 3, 6)
```

- `index_dims = [0]`, shape = (2,)

- Sliced dim 1 = 1:4 → size 3
 - Remaining dim 2 = 6
 - **Set shape** = (2, 3, 6)
-

Implementation Details

General Validation

Explanations
Number of input tensors must be 2 tensors: <code>input</code> (0) and <code>value</code> (1)
Do not allow setitem with scalar tensors (<code>rank == 0</code>)
Input (as value) must be on device and allocated
Input must be INTERLEAVED
Value must has memory configurations same as input: • Rank • Data type • Tensor layout • Memory layout
Number of index tensors (if available) must has corresponding setting rank
All indexing tensors (if available) must be: • On device and allocated • Has rank of 1 • ROW_MAJOR_LAYOUT and INTERLEAVED • Data type: <code>int32</code>

Row-Major Interleaved Inputs, Output

This version of implementation is tightly integrated with TTNN and supports:

- `INTERLEAVED` and `ROW_MAJOR_LAYOUT` layout for all tensors

- Datatype flexible but `bfloat8_b` since it requires `TILE_LAYOUT` .

Validation & Output Spec

Input memory configuration:

- **Layout:** `ROW_MAJOR`
- **Memory layout:** `INTERLEAVED`
- **Data Type:** `Any`

Input shapes: `Any` (`rank > 0` - not scalar)

It's worth noting that after take slice from input tensor (based on slice indices), the **slice input tensor and value tensor must has the same shape.**

Output:

- Memory configuration: `ROW_MAJOR` , `INTERLEAVED`
- Datatype: as input datatype

Support Constraints:

- **Input rank > 0**
- Setitem with **slices**: broadcastable
- Setitem with indexing tensors: broadcastable

Pipeline

[#todo] host logic and kernel logic diagram

Reader Stage

Compile-time Arguments

Name	Type	Description
<code>input_cbi</code>	<code>uint32_t</code>	Circular buffer index for the input tensor.
<code>value_cbi</code>	<code>uint32_t</code>	Circular buffer index for the value tensor.

<code>input_is_dram</code>	<code>bool</code>	Indicates whether the input tensor is in DRAM.
<code>value_is_dram</code>	<code>bool</code>	Indicates whether the value tensor is in DRAM.
<code>inner_slices_size</code>	<code>uint32_t</code>	Size (in elements) of each inner slice written per iteration.
<code>input_shape_last_dim</code>	<code>uint32_t</code>	Size of the last dimension in the input tensor.
<code>value_shape_last_dim</code>	<code>uint32_t</code>	Size of the last dimension in the value tensor.
<code>tensor_rank</code>	<code>uint32_t</code>	Rank of both input and value tensors.
<code>element_size</code>	<code>uint32_t</code>	Size in bytes of each element.
<code>is_broadcast_last_dim</code>	<code>bool</code>	Whether the last dimension of value is broadcast across input.
<code>n_ttnn_tensor</code>	<code>uint32_t</code>	Number of additional index tensors.

Runtime Arguments

Name	Type	Description
<code>input_address</code>	<code>uint32_t</code>	Base address of the input tensor in memory.
<code>value_address</code>	<code>uint32_t</code>	Base address of the value tensor in memory.

<code>start_id</code>	<code>uint32_t</code>	Global start stick ID for this core's slice.
<code>num_sticks_per_core</code>	<code>uint32_t</code>	Number of sticks this core will process.
<code>input_shape</code>	<code>vector<uint32_t></code>	Full shape of the input tensor.
<code>input_set_shape</code>	<code>vector<uint32_t></code>	Shape of the region being updated inside input tensor.
<code>value_shape</code>	<code>vector<uint32_t></code>	Shape of the value tensor.
<code>begins</code>	<code>vector<uint32_t></code>	Start indices for the slice in each dimension.
<code>steps</code>	<code>vector<uint32_t></code>	Step size for slicing per dimension.
<code>is_ttnn_tensor</code>	<code>vector<uint32_t></code>	Check if that dim contains ttnn::Tensor or not.
<code>indexes_address</code>	<code>vector<uint32_t></code>	Base address of the index tensors in memory.

<code>indexes_page_size</code>	<code>vector<uint32_t></code>	Page size of the index tensors in memory.
<code>indexes_is_dram</code>	<code>vector<uint32_t></code>	Buffer type of the index tensors in memory.

Writer Stage

Compile-time Arguments

Name	Type	Description
<code>input_cbi</code>	<code>uint32_t</code>	Circular buffer index for the input tensor.
<code>input_is_dram</code>	<code>bool</code>	Indicates whether the input tensor resides in DRAM.
<code>input_shape_last_dim</code>	<code>uint32_t</code>	Size of the last dimension in the input tensor.
<code>element_size</code>	<code>uint32_t</code>	Size in bytes of each tensor element.
<code>tensor_rank</code>	<code>uint32_t</code>	Rank of the input tensor.
<code>n_ttnn_tensor</code>	<code>uint32_t</code>	Number of additional index tensors.

Runtime Arguments

Name	Type	Description
------	------	-------------

<code>input_addr ess</code>	<code>uint32_t</code>	Base memory address of the input tensor.
<code>start_id</code>	<code>uint32_t</code>	Local core start stick ID for writing.
<code>num_sticks _per_core</code>	<code>uint32_t</code>	Number of sticks to write per core.
<code>input_shape</code>	<code>vector<u int32_t></code>	Full shape of the input tensor.
<code>input_set_ shape</code>	<code>vector<u int32_t></code>	Shape of the region being updated.
<code>begins</code>	<code>vector<u int32_t></code>	Start indices for the slice.
<code>steps</code>	<code>vector<u int32_t></code>	Step sizes for each slice dimension.
<code>is_ttnn_ tensor</code>	<code>vector<u int32_t></code>	Check if that dim contains ttnn::Tensor or not.
<code>indexes_ad dress</code>	<code>vector<u int32_t></code>	Base address of the index tensors in memory.
<code>indexes_pa ge_size</code>	<code>vector<u int32_t></code>	Page size of the index tensors in memory.
<code>indexes_is _dram</code>	<code>vector<u int32_t></code>	Buffer type of the index tensors in memory.

Pseudo code

Readers

▼ `setter_reader_unpad_dims_rm_interleaved_start_id.cpp`

This kernel is called in general case where tensors are INTERLEAVED, ROW-MAJOR, non-strided slice for inner dimension. Overall pipeline: **load args** → **compute indices** → **read memory** → **broadcast or copy** → **commit write**

```
1 function READER_KERNEL():
2     // === Load runtime arguments ===
3     input_addr, value_addr, start_id, num_sticks ← get_args(0..3)
4     input_shape_ptr ← get_arg_ptr(4)
5
6     // === Load metadata pointers ===
7     input_shape ← input_shape_ptr
8     input_set_shape ← input_shape + rank
9     value_shape ← input_set_shape + rank
10    begins, steps ← value_shape + rank, value_shape + 2*rank
11    is_index_tensor ← steps + rank
12    index_tensors ← load_index_tensors(n_tensors, value_cbi + 1)
13
14    // === Setup memory layouts ===
15    input_layout ← layout(input_addr, input_shape[-1], input_is_dram)
16    value_layout ← layout(value_addr, value_shape[-1], value_is_dram)
17    value_ptr ← get_read_ptr(value_cbi)
18    offset ← begins[last_dim] * element_size
19
20    // === Main loop per stick ===
21    for stick_id in range(start_id, start_id + num_sticks):
22        // Compute memory location based on index or slice
23        input_bank ← find_input_bank_id(stick_id, ...)
24        value_bank ← find_value_bank_id(stick_id, ...)
25
26        reserve_cb(input_cbi)
27        input_ptr ← get_read_ptr(input_cbi)
28
29        // Load data into L1
30        async_read(input_layout, input_bank, input_ptr)
31        async_read(value_layout, value_bank, value_ptr)
32        wait_reads()
33
34        if is_broadcast_last_dim:
35            val ← read_scalar(value_ptr)
36            dst ← cast(input_ptr, element_size)
37            if is_index_tensor[last_dim]:
38                broadcast_to_indices(dst, index_tensors[-1], val)
39            else:
40                broadcast_to_strided(dst, begins[last_dim], steps[last_dim], val)
41        else:
42            copy(input_ptr + offset, value_ptr, value_shape[-1] * element_size)
43
44        push_cb(input_cbi)
```

▼ strided_reader_rm_interleaved_nd.cpp

This kernel is called in general case where tensors are INTERLEAVED, ROW-MAJOR, strided slice for inner dimension.

```
1 function READER_KERNEL():
2     // === Load arguments and metadata ===
```

```

3  input_addr, value_addr, start_id, num_sticks ← get_args(0..3)
4  input_shape_ptr ← get_arg_ptr(4)
5
6  // === Load tensor shape and indexing config ===
7  input_shape      ← input_shape_ptr
8  input_set_shape  ← input_shape + rank
9  value_shape      ← input_set_shape + rank
10 begins, steps    ← value_shape + rank, value_shape + 2*rank
11 is_index_tensor  ← steps + rank
12 index_tensors    ← load_index_tensors(n_tensors, value_cbi + 1)
13
14 // === Setup page layouts ===
15 input_layout ← layout(input_addr, input_shape[-1], input_is_dram)
16 value_layout ← layout(value_addr, value_shape[-1], value_is_dram)
17 value_ptr ← get_read_ptr(value_cbi)
18
19 // === For each stick in assigned range ===
20 for stick_id in start_id .. start_id + num_sticks:
21     input_bank ← find_input_bank_id(...)
22     value_bank ← find_value_bank_id(...)
23
24     reserve_cb(input_cbi)
25     input_ptr ← get_read_ptr(input_cbi)
26
27     async_read(input_layout, input_bank, input_ptr)
28     async_read(value_layout, value_bank, value_ptr)
29     wait_reads()
30
31     if is_broadcast_last_dim:
32         // Case: scalar or (1,) → broadcast across last dim
33         val ← read_scalar(value_ptr)
34         dst ← cast(input_ptr, element_size)
35         if is_index_tensor[last_dim]:
36             broadcast_to_indices(dst, index_tensors[-1], val)
37         else:
38             broadcast_to_strided(dst, begins[last_dim], steps[last_dim], val)
39     else:
40         // Case: real value tensor (not scalar) → write selected elements
41         val_buf ← cast(value_ptr, element_size)
42         dst ← cast(input_ptr, element_size)
43         if is_index_tensor[last_dim]:
44             assign_to_indices(dst, index_tensors[-1], val_buf)
45         else:
46             assign_to_strided(dst, begins[last_dim], steps[last_dim], val_buf)
47
48     push_cb(input_cbi)

```

Writers

✓ setter_writer_stick_layout_interleaved_start_id.cpp

This kernel is called in general case where tensors are INTERLEAVED, ROW-MAJOR.

```

1  function WRITER_KERNEL():
2      // === Load runtime arguments ===
3      input_addr, start_id, num_sticks ← get_args(0..2)
4      input_shape_ptr ← get_arg_ptr(3)
5

```

```

6 // === Load shape metadata ===
7 input_shape ← input_shape_ptr
8 input_set_shape ← input_shape + rank
9 begins, steps ← input_set_shape + rank, input_set_shape + 2*rank
10 is_index_tensor ← steps + rank
11 index_tensors ← load_index_tensors(n_ttnn_tensor, input_cbi + 2)
12
13 // === Setup memory layout ===
14 input_layout ← layout(input_addr, input_shape_last_dim, input_is_dram)
15
16 // === Loop over assigned sticks ===
17 for stick_id in range(start_id, start_id + num_sticks):
18     wait_for_cb(input_cbi)
19     l1_ptr ← get_write_ptr(input_cbi)
20
21     // Compute flat memory location to write to
22     bank_id ← find_input_bank_id(
23         stick_id, input_shape, input_set_shape,
24         begins, steps, is_index_tensor, index_tensors,
25         n_ttnn_tensor, tensor_rank
26     )
27
28     // Write back from L1 → global memory
29     noc_write(l1_ptr, address(bank_id, input_layout), input_page_size)
30     wait_for_write_complete()
31     pop_cb(input_cbi)

```

Memory Usage

[#todo]

TILE Interleaved I/O

Validation & Output Spec

Input memory configuration:

- Layout: TILE
- Memory layout: INTERLEAVED
- Data Type: Any

Input shapes: Any (rank > 0)

It's worth noting that after take slice from input tensor (based on slice indices), the **slice input tensor and value tensor must has the same shape.**

Output:

- Memory configuration: **TILE** , **INTERLEAVED**
- Datatype: as input datatype

Support Constraints:

• Input Requirements

- The input tensor must have **rank ≥ 1** (i.e., at least one dimension).
- The input and value tensors must have **same settings** (i.e., data format, layout, ...).

• Slicing Constraints (on the last two dimensions)

- Only **slice-based indexing** is supported on the last two dimensions.
- Each slice must satisfy either of the following:
 - The **start and end** of the slice align exactly with a **full tile** (e.g., **32 $\rightarrow$ 64** or **32 $\rightarrow$ 128**), or
 - The **slice shape** must **fully fit** the input shape.
- The **step size must be 1**.
- If any of these conditions are violated, the operation will **fall back to row-major** and be processed as described in the previous section.
- TILE layout is also supported broadcasting, but only in case the last 2 dimensions must be match with the shape of last 2 dimensions of input.

Reader Stage

Compile-time Arguments

Name	Type	Description
value_cbi	uint 32_t	Circular buffer index for the value tensor.
value_is_dram	bool	Indicates whether the value tensor is in DRAM.
tensor_rank	uint 32_t	Rank of both input and value tensors.
n_ttnn_tensor	uint 32_t	Number of additional index tensors.

Runtime Arguments

Name	Type	Description
<code>input_address</code>	<code>uint32_t</code>	Base address of the input tensor in memory.
<code>value_address</code>	<code>uint32_t</code>	Base address of the value tensor in memory.
<code>start_id</code>	<code>uint32_t</code>	Global start stick ID for this core's slice.
<code>num_tiles_per_core</code>	<code>uint32_t</code>	Number of tiles this core will process.
<code>input_shape</code>	<code>vector<uint32_t></code>	Full shape of the input tensor.
<code>input_set_shape</code>	<code>vector<uint32_t></code>	Shape of the region being updated inside input tensor.
<code>value_shape</code>	<code>vector<uint32_t></code>	Shape of the value tensor.
<code>begins</code>	<code>vector<uint32_t></code>	Start indices for the slice in each dimension.
<code>steps</code>	<code>vector<uint32_t></code>	Step size for slicing per dimension.

<code>is_ttnn_tensor</code>	<code>vector<uint32_t></code>	Check if that dim contains ttnn::Tensor or not.
<code>indexes_address</code>	<code>vector<uint32_t></code>	Base address of the index tensors in memory.
<code>indexes_page_size</code>	<code>vector<uint32_t></code>	Page size of the index tensors in memory.
<code>indexes_is_dram</code>	<code>vector<uint32_t></code>	Buffer type of the index tensors in memory.

Writer Stage

Compile-time Arguments

Name	Type	Description
<code>value_cbi</code>	<code>uint32_t</code>	Circular buffer index for the value tensor.
<code>input_is_dram</code>	<code>bool</code>	Indicates whether the input tensor resides in DRAM.
<code>tensor_rank</code>	<code>uint32_t</code>	Rank of both input and value tensors.
<code>n_ttnn_tensor</code>	<code>uint32_t</code>	Number of additional index tensors.

Runtime Arguments

Name	Type	Description
------	------	-------------

<code>input_address</code>	<code>uint32_t</code>	Base memory address of the input tensor.
<code>start_id</code>	<code>uint32_t</code>	Local core start stick ID for writing.
<code>num_tiles_per_core</code>	<code>uint32_t</code>	Number of sticks to write per core.
<code>input_shape</code>	<code>vector<uint32_t></code>	Full shape of the input tensor.
<code>input_set_shape</code>	<code>vector<uint32_t></code>	Shape of the region being updated.
<code>begins</code>	<code>vector<uint32_t></code>	Start indices for the slice.
<code>steps</code>	<code>vector<uint32_t></code>	Step size for slicing per dimension.
<code>is_ttnn_tensor</code>	<code>vector<uint32_t></code>	Check if that dim contains ttnn::Tensor or not.
<code>indexes_address</code>	<code>vector<uint32_t></code>	Base address of the index tensors in memory.
<code>indexes_page_size</code>	<code>vector<uint32_t></code>	Page size of the index tensors in memory.
<code>indexes_is_dram</code>	<code>vector<uint32_t></code>	Buffer type of the index tensors in memory.

Performance Evaluation

Objective

The goal of this benchmark is to compare the latency and correctness of two methods for modifying subregions of TTNN tensors:

1. **In-place Setitem (`tensor[indices] = value`)**: Direct assignment using `ttnn.__setitem__`.
2. **Concat-Based Workaround (`tensor_rank_concat`)**: A slice-then-concat strategy to reconstruct tensors with partial replacements in one or more dimensions.

This comparison helps determine whether the native in-place operator is more performant or whether a fallback concat-based method is preferable in constrained environments (e.g., where `__setitem__` may not be natively supported by TT-METAL).

Methodology

Each test case initializes a 2D or 4D tensor using `ttnn.from_torch()` and defines a slicing region and replacement tensor. Two functions are used for benchmarking:

- `measure_setitem()` : Clones the tensor, applies in-place assignment, and measures time.
- `measure_concat()` : Clones the tensor, reconstructs it using slicing and `ttnn.concat`, and measures time.

Both methods run for `num_runs = 100` and average latency is recorded.

▼ Concat-Based Workaround Pseudocode

```
1  Function tensor_rank_concat(tensor, indices, value):
2
3      1. Normalize indices:
4          - Extend the 'indices' tuple to match tensor's rank by appending full slices
            (slice(None)) as needed.
5
6      2. Identify partial slice dimensions:
7          - For each dimension in the normalized indices:
8              - If index is a slice with start > 0 or stop < tensor.shape[dim]:
9                  - Mark as a dimension that requires concat-based slicing.
10
11     3. If no such dimensions found:
12         - Perform direct in-place assignment:
13             tensor[indices] = value
14             return tensor
15
16     4. Forward slicing:
```

```

17 - Initialize current_tensor = tensor
18 - For each slice_dim in identified dimensions:
19     a. Slice 'before' region: from 0 to slice.start (if applicable)
20     b. Slice 'after' region: from slice.stop to end (if applicable)
21     c. Update current_tensor to the 'middle' slice (slice.start:slice.stop)
22     d. Store 'before' and 'after' parts in a dictionary
23
24 5. Replace current_tensor with the 'value' tensor.
25
26 6. Backward reconstruction:
27     - For each slice_dim in reverse order:
28         a. Initialize parts_to_concat = []
29         b. Append 'before' part (if exists)
30         c. Append the current_tensor (i.e., the replaced middle)
31         d. Append 'after' part (if exists)
32         e. Concatenate all parts along the current dimension
33         f. Set current_tensor = concatenated result
34
35 7. Return the fully reconstructed tensor.
36

```

Testcases

Case	Shape	Slice Indices	Value Shape
Case 1	(1, 2, 256, 256)	(slice(None), slice(None), slice(32, 128), slice(64, 192))	(1, 2, 96, 128)
Case 2	(2, 128, 128, 64)	(slice(0, 1), slice(32, 96), slice(32, 64),	(1, 64, 32, 64)

		slice(None)	
Case 3	(4, 64, 128, 128)	(slice(1, 3), slice(16, 48), slice(32, 96), slice(0, 64))	(2, 32, 64, 64)
Case 4	(1024, 1024)	(slice(256 , 768), slice(256, 768))	(512, 512)

Runtime Benchmark

No.	Operation	Time/Iter (ms)	Speedup ratio Concat / Setitem (TILE)
1	In-place Setitem (RM)	0.564	7.21
	In-place Setitem (TILE)	0.679	
	Concat-Based Workaround	4.898	
2	In-place Setitem (RM)	1.055	14.58
	In-place Setitem (TILE)	0.545	
	Concat-Based Workaround	7.944	
3	In-place Setitem (RM)	1.263	21.53

4	In-place Setitem (TILE)	0.539	7.47
	Concat-Based Workaround	11.603	
	In-place Setitem (RM)	0.705	
	In-place Setitem (TILE)	0.672	
	Concat-Based Workaround	5.017	

Smoke testing

To test `ttnn.setitem` operation, move to branch `dev-v0.59` of `BOS_TT_METAL` (Commit: [tests for setitem row_major, interleaved \(4f77179d\) · Commits · NPU_Team / BOS_TT_METAL · GitLab](#)).

Run test for `ROW_MAJOR_LAYOUT`, `INTERLEAVED` by bellow command:

```
1 pytest $TT_METAL_HOME/tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py
```

Expected output:

```

PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_invalid_input_count
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_invalid_value_dtype[inp.dtype=torch.int64-value.dtype=torch.float32]
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_invalid_value_dtype[inp.dtype=torch.int32-value.dtype=torch.float32]
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_invalid_value_dtype[inp.dtype=torch.int64-value.dtype=torch.bfloat16]
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_unbroadcastable_check[inp_shape=[3, 5]-value_shape=[2, 5]]
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_unbroadcastable_check[inp_shape=[4, 5]-value_shape=[2, 5]]
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_unbroadcastable_check[inp_shape=[4, 5]-value_shape=[5, 5]]
PASSED tests/ttnn/unit_tests/operations/test_setitem_device_op_rm.py::test_setitem_index_length_exceeds
===== 64 passed in 16.93s =====

```

References

- 🔥 [Slicing, Indexing, and Masking — tensordict main documentation](#)
- [TT-Metal GitHub](#)